2.QR code creation and recognition

2.1、QR code

2.1.1、QR Code Introduction

QR code is a type of two-dimensional barcode, which comes from the abbreviation of "Quick Response" in English, meaning quick response. It originated from the inventor's hope that QR codes can quickly decode their content. QR codes not only have large information capacity, high reliability, and low cost, but also can represent various textual information such as Chinese characters and images. They have strong confidentiality and anti-counterfeiting capabilities, and are very convenient to use. More importantly, QR code technology is open-source.

2.1.2、QR codes Characteristics

The data values in QR codes contain duplicate information (redundant values). Therefore, even if up to 30% of the QR code structure is damaged, it does not affect the readability of the QR code. The storage space of QR codes can reach up to 7089 bits or 4296 characters, including punctuation and special characters, which can be written into the QR code. In addition to numbers and characters, words and phrases (such as website addresses) can also be encoded. As more data is added to the QR code, the code size increases and the code structure becomes more complex.

2.1.3、QR code creation and recognition

1) Create QR code

- Code path

  ```
  ~/orbbec_ws/src/yahboomcar_visual/simple_qrcode/QRcode_Create.py
  ```

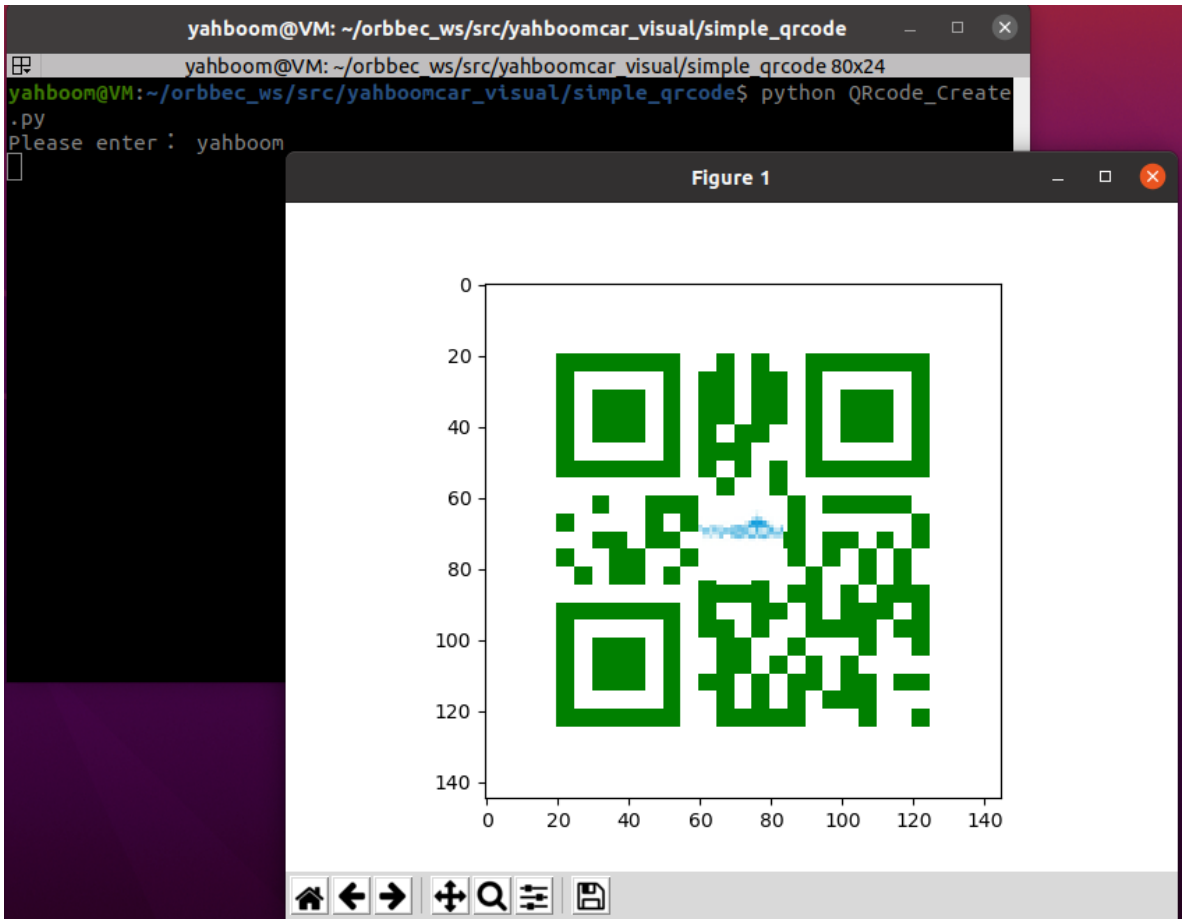
- Installation package

  ```
  python3 -m pip install qrcode pyzbar
  sudo apt-get install libzbar-dev
  ```

- Start

  ```
  cd ~/orbbec_ws/src/yahboomcar_visual/simple_qrcode
  python QRcode_Create.py
  ```

  After the program runs, it will prompt for the generated content, and the enter key will confirm the content. Here, taking the creation of the "yahobom" string as an example.



- Core source code analysis

```
#Create qrcode object
qr = qrcode.QRCode(
    version=1,
    error_correction=qrcode.constants.ERROR_CORRECT_H,
    box_size=5,
    border=4,)
#各参数含义
'''version：值为1~40的整数，控制二维码的大小（最小值是1，是个12×12的矩阵）。
如果想让程序自动确定，将值设置为 None 并使用 fit 参数即可。
error_correction：控制二维码的错误纠正功能。可取值下列4个常量。
ERROR_CORRECT_L：大约7%或更少的错误能被纠正。
ERROR_CORRECT_M（默认）：大约15%或更少的错误能被纠正。
ROR_CORRECT_H：大约30%或更少的错误能被纠正。
box_size：控制二维码中每个小格子包含的像素数。
border：控制边框（二维码与图片边界的距离）包含的格子数（默认为4，是相关标准规定的最小值）'''
#qrcode二维码添加logo
#Meaning of each parameter
'''Version: An integer ranging from 1 to 40 that controls the size of the QR code (with a minimum value of 1, it is a 12 × 12 matrix).
If you want the program to automatically determine, set the value to None and use the fit parameter.
Error Correction: Controls the error correction function of the QR code. The following four constants can be taken as values.
VNet CORD_L: Approximately 7% or less of errors can be corrected.
VNet CORD.M (default): Approximately 15% or less of errors can be corrected.
```

```
ROR-CORRECT-H: Approximately 30% or less of errors can be corrected.
Box_2: Controls the number of pixels contained in each small cell of the QR
code.
Border: controls the number of cells contained in the border (the distance
between the QR code and the image boundary) (default is 4, which is the minimum
value specified by relevant standards)'''
#QR Code QR Code Add Logo
my_file = Path(logo_path)
if my_file.is_file(): img = add_logo(img, logo_path)
#Add data
qr.add_data(data)
# fill data
qr.make(fit=True)
# Generate images
img = qr.make_image(fill_color="green", back_color="white")
```

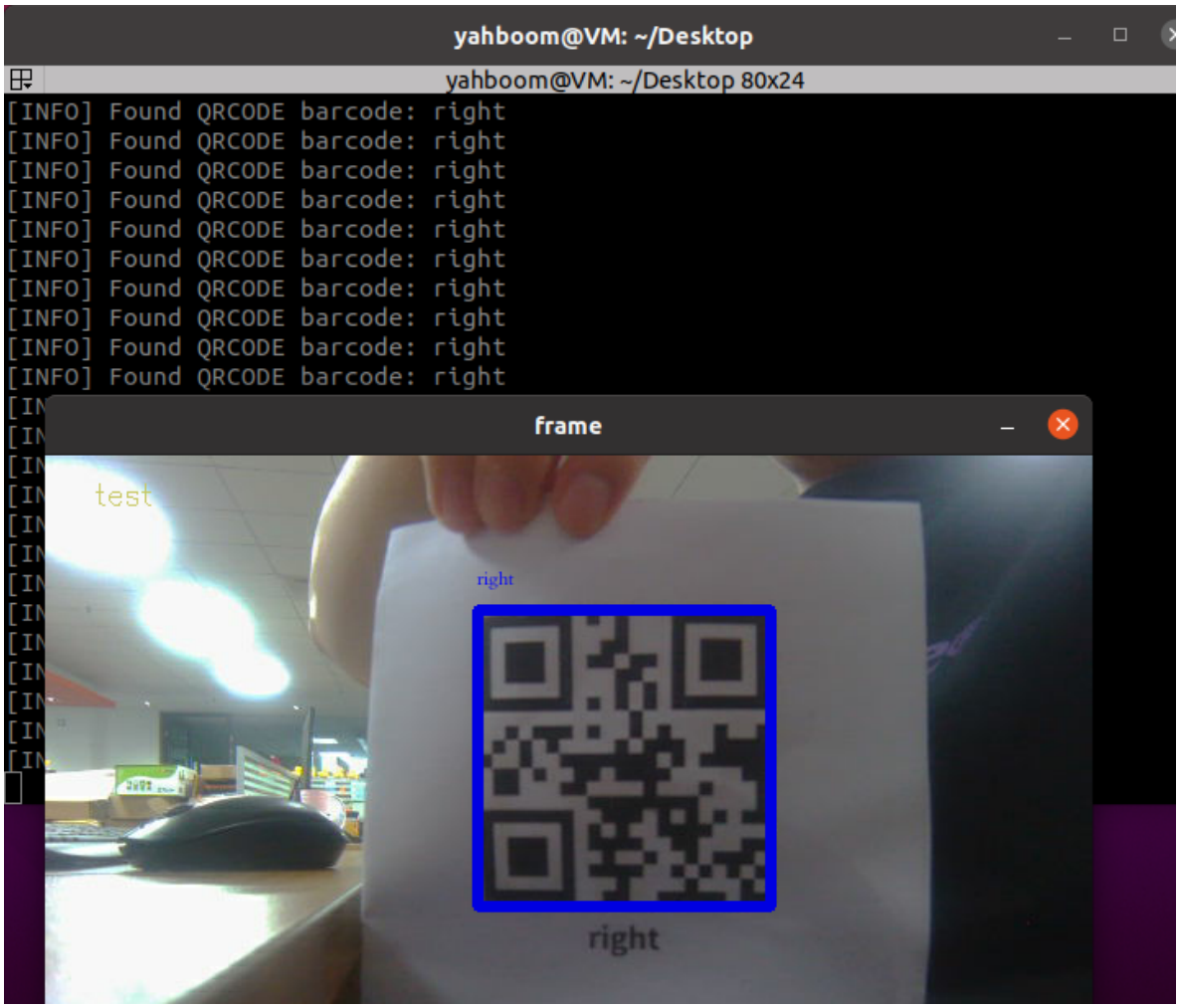
2) Identify QR code

- Code path

```
~/orbbec_ws/src/yahboomcar_visual/yahboomcar_visual/QRcode_Parsing.py
```

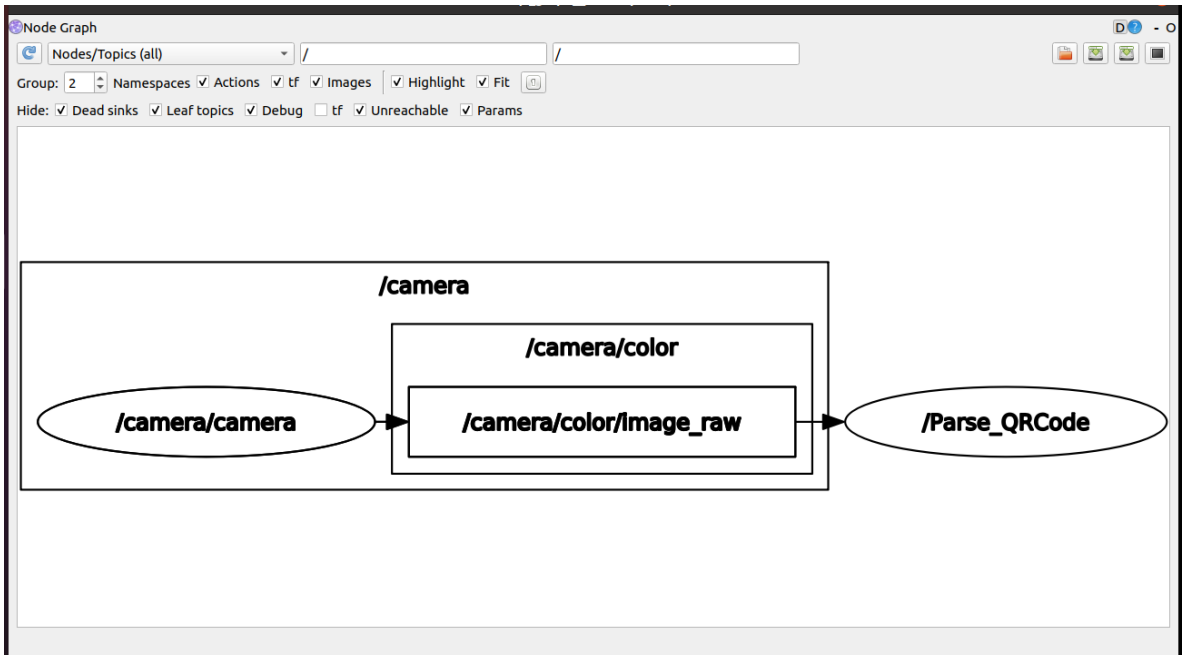
- Start

```
ros2 launch astra_camera gemini.launch.xml
ros2 run yahboomcar_visual QRcode_Parsing
```



- View node communication diagram

```
ros2 run rqt_graph rqt_graph
```



- Core source code analysis

```
#Subscribe to color image information
self.sub_img =
self.create_subscription(CamImage,'/camera/color/image_raw',self.handleTopic,100
)
#Passing image data to the parsing image function in the callback function
frame = self.decodeDisplay(frame)
#Analyze images
def decodeDisplay(self,image):
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    # The output Chinese characters need to be converted to Unicode encoding
first
    barcodes = pyzbar.decode(gray)
    for barcode in barcodes:
        # Extract the position of the boundary box of the TWO-DIMENSIONAL code
        # Draw the bounding box for the bar code in the image
        (x, y, w, h) = barcode.rect
        cv.rectangle(image, (x, y), (x + w, y + h), (225, 0, 0), 5)
        encoding = 'UTF-8'
        # to draw it, you need to convert it to a string
        barcodeData = barcode.data.decode(encoding)
        barcodeType = barcode.type
        # Draw the data and type on the image
        pilimg = Image.fromarray(image)
        # create brush
        draw = ImageDraw.Draw(pilimg)  # Print on picture
        # parameter 1: font file path, parameter 2: font size
        fontStyle =
ImageFont.truetype("/home/yahboom/orbbec_ws/src/yahboomcar_visual/yahboomcar_vis
ual/Block_Simplified.TTF", size=12, encoding=encoding)
        # Parameter 1: print coordinates, parameter 2: text, parameter 3: font
color, parameter 4: font
        draw.text((x, y - 25), str(barcode.data, encoding), fill=(255, 0, 0),
font=fontStyle)
        # PIL picture to CV2 picture
        image = np.array(pilimg)
        # Print barcode data and barcode type to terminal
        print("[INFO] Found {} barcode: {}".format(barcodeType, barcodeData))
        return image
```